

CS787: GENERATIVE ARTIFICIAL INTELLIGENCE RECURRENT AND CONVOLUTIONAL NEURAL NETWORKS

Arnab Bhattacharya
`arnabb@cse.iitk.ac.in`

Computer Science and Engineering,
Indian Institute of Technology, Kanpur
<http://web.cse.iitk.ac.in/~cs787/>

1st semester, 2026-27

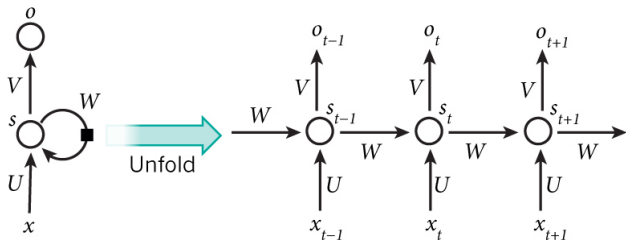
Wed 12:00–13:15 at L20, Fri 12:00–13:15 at L19

Recurrent Neural Networks

- Recurrent Neural Networks (RNNs) model recurring patterns
 - Same task is repeated for every element of a sequence

Recurrent Neural Networks

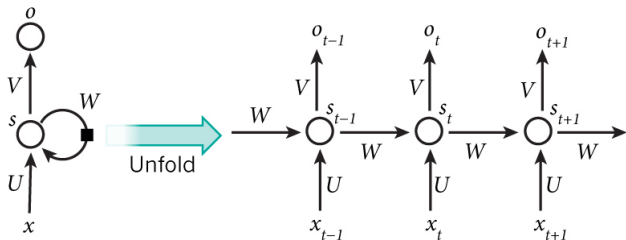
- **Recurrent Neural Networks (RNNs)** model recurring patterns
 - Same task is repeated for every element of a sequence
- Hidden nodes are *not* independent of each other



- Output depends on previous steps, i.e., it uses “memory”

Recurrent Neural Networks

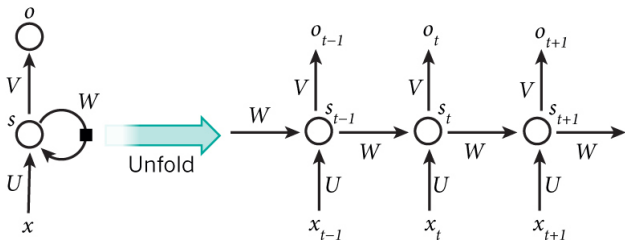
- **Recurrent Neural Networks (RNNs)** model recurring patterns
 - Same task is repeated for every element of a sequence
- Hidden nodes are *not* independent of each other



- Output depends on previous steps, i.e., it uses “memory”
- “Unrolling” or “unfolding” produces the layers
- If a 3-length *context* is needed, RNN is unfolded to 3 layers

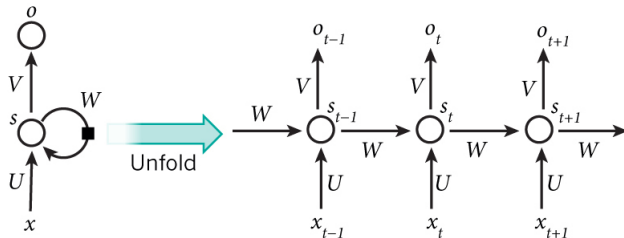
Recurrent Neural Networks

- **Recurrent Neural Networks (RNNs)** model recurring patterns
 - Same task is repeated for every element of a sequence
- Hidden nodes are *not* independent of each other



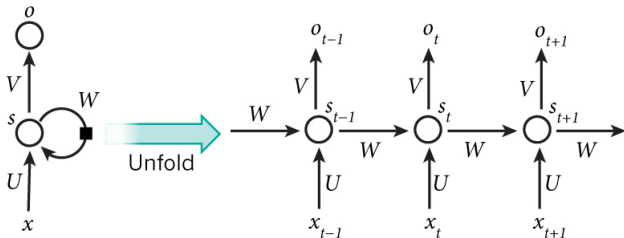
- Output depends on previous steps, i.e., it uses “memory”
- “Unrolling” or “unfolding” produces the layers
- If a 3-length *context* is needed, RNN is unfolded to 3 layers
- Same parameters U, V, W are shared across the layers
 - General deep networks are not constrained by this

Components of an RNN



- $\vec{x}$ at each step is the *one-hot* vector (i.e., only 1 element is on)

Components of an RNN



- $\vec{x}$ at each step is the *one-hot* vector (i.e., only 1 element is on)
- $\vec{s}_t$ is “memory” as it captures everything previous

$$\vec{s}_t = f(U \cdot \vec{x}_t + W \cdot \vec{s}_{t-1} + \vec{b}_s)$$

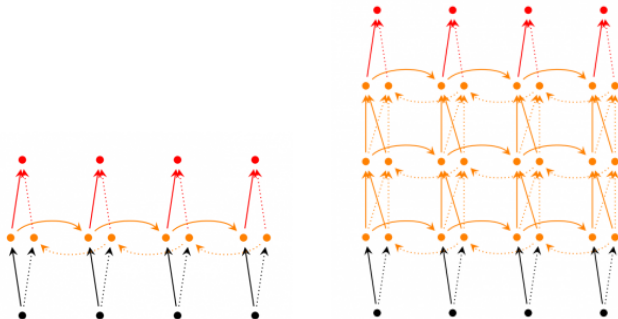
- f is a non-linear function such as sigmoid or hyperbolic tangent
- $\vec{o}_t$ is *output* at step t

$$\vec{o}_t = g(V \cdot \vec{s}_t + \vec{b}_o)$$

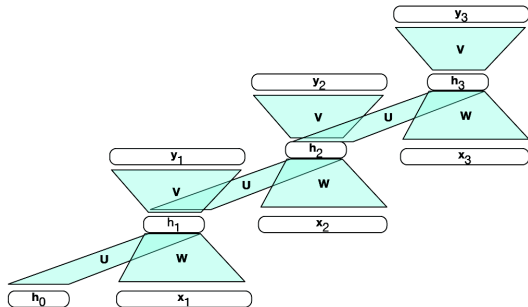
- Generally, g is the *softmax* function to produce distributions

Types of RNNs

- **Bi-directional RNNs** use future as well as past to model present
- **Deep/stacked bi-directional RNNs** use multiple layers per time step

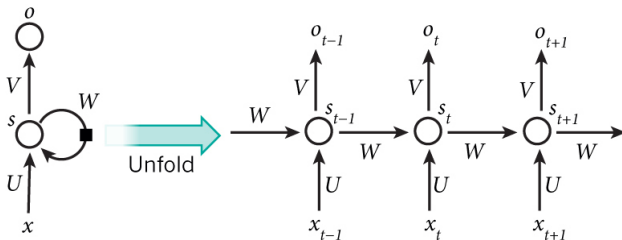


Training RNNs



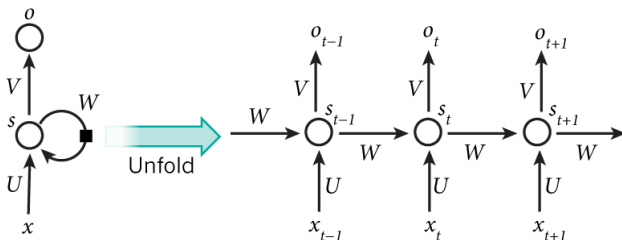
- Simple *backpropagation* does not work since there are loops
- Unfolding removes loops
- Backpropagation is then adopted as **backpropagation through time (BPTT)**

Backpropagation through Time (BPTT)



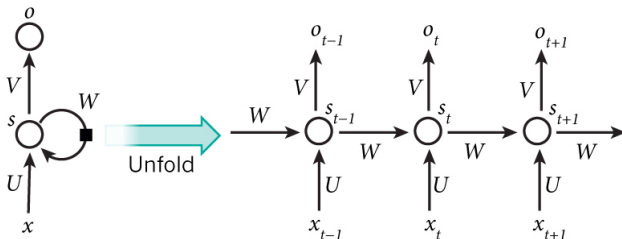
- BPTT unfolds, and then applies backpropagation at each time step

Backpropagation through Time (BPTT)



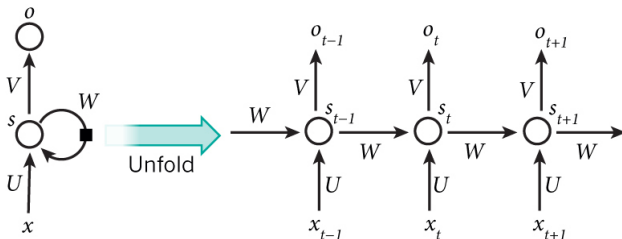
- BPTT unfolds, and then applies backpropagation at each time step
- Total loss L is sum of losses L_i across t timesteps

Backpropagation through Time (BPTT)



- BPTT unfolds, and then applies backpropagation at each time step
- Total loss L is sum of losses L_i across t timesteps
- Gradient is computed for each time step i

Backpropagation through Time (BPTT)

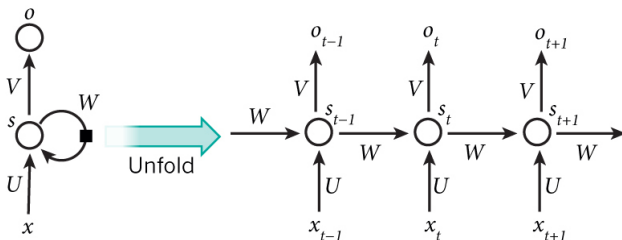


- BPTT unfolds, and then applies backpropagation at each time step
- Total loss L is sum of losses L_i across t timesteps
- Gradient is computed for each time step i

$$\frac{\partial L_i}{\partial W} = \frac{\partial L_i}{\partial y_i} \cdot \frac{\partial y_i}{\partial h_i} \cdot \frac{\partial h_i}{\partial W}$$

- Enough for one time step

Backpropagation through Time (BPTT)



- BPTT unfolds, and then applies backpropagation at each time step
- Total loss L is sum of losses L_i across t timesteps
- Gradient is computed for each time step i

$$\frac{\partial L_i}{\partial W} = \frac{\partial L_i}{\partial y_i} \cdot \frac{\partial y_i}{\partial h_i} \cdot \frac{\partial h_i}{\partial W}$$

- Enough for one time step
- For multiple time steps, *total gradient* is accumulated across time steps

Total Gradient

- Not just simple summation since

Total Gradient

- Not just simple summation since hidden states h_i depend on previous hidden states h_{i-1}
- For two states

$$\begin{aligned}\frac{\partial L}{\partial W} &= \left(\frac{\partial L}{\partial W} \right)_i + \left(\frac{\partial L}{\partial W} \right)_{i-1} \\ &= \frac{\partial L_i}{\partial y_i} \cdot \frac{\partial y_i}{\partial h_i} \cdot \frac{\partial h_i}{\partial W} + \frac{\partial L_i}{\partial y_i} \cdot \frac{\partial y_i}{\partial h_i} \cdot \left(\frac{\partial h_i}{\partial h_{i-1}} \right) \cdot \frac{\partial h_{i-1}}{\partial W}\end{aligned}$$

Total Gradient

- Not just simple summation since hidden states h_i depend on previous hidden states h_{i-1}
- For two states

$$\begin{aligned}\frac{\partial L}{\partial W} &= \left(\frac{\partial L}{\partial W} \right)_i + \left(\frac{\partial L}{\partial W} \right)_{i-1} \\ &= \frac{\partial L_i}{\partial y_i} \cdot \frac{\partial y_i}{\partial h_i} \cdot \frac{\partial h_i}{\partial W} + \frac{\partial L_i}{\partial y_i} \cdot \frac{\partial y_i}{\partial h_i} \cdot \left(\frac{\partial h_i}{\partial h_{i-1}} \right) \cdot \frac{\partial h_{i-1}}{\partial W}\end{aligned}$$

- Successive timestamps have more partial derivatives

Total Gradient

- Not just simple summation since hidden states h_i depend on previous hidden states h_{i-1}
- For two states

$$\begin{aligned}\frac{\partial L}{\partial W} &= \left(\frac{\partial L}{\partial W} \right)_i + \left(\frac{\partial L}{\partial W} \right)_{i-1} \\ &= \frac{\partial L_i}{\partial y_i} \cdot \frac{\partial y_i}{\partial h_i} \cdot \frac{\partial h_i}{\partial W} + \frac{\partial L_i}{\partial y_i} \cdot \frac{\partial y_i}{\partial h_i} \cdot \left(\frac{\partial h_i}{\partial h_{i-1}} \right) \cdot \frac{\partial h_{i-1}}{\partial W}\end{aligned}$$

- Successive timestamps have more partial derivatives
- Overall,

$$\frac{\partial L}{\partial W} = \sum_{i=1}^t \left[\frac{\partial L_i}{\partial y_i} \cdot \frac{\partial y_i}{\partial h_i} \cdot \left(\prod_{j=i}^{i-1} \frac{\partial h_j}{\partial h_{j-1}} \right) \cdot \frac{\partial h_i}{\partial W} \right]$$

Other Weight Matrices

- Input weight matrix U is updated in the same way

$$\frac{\partial L}{\partial U} = \sum_{i=1}^t \left[\frac{\partial L_i}{\partial y_i} \cdot \frac{\partial y_i}{\partial h_i} \cdot \left(\prod_{j=t}^{i-1} \frac{\partial h_j}{\partial h_{j-1}} \right) \cdot \frac{\partial h_i}{\partial U} \right]$$

Other Weight Matrices

- Input weight matrix U is updated in the same way

$$\frac{\partial L}{\partial U} = \sum_{i=1}^t \left[\frac{\partial L_i}{\partial y_i} \cdot \frac{\partial y_i}{\partial h_i} \cdot \left(\prod_{j=t}^{i-1} \frac{\partial h_j}{\partial h_{j-1}} \right) \cdot \frac{\partial h_i}{\partial U} \right]$$

- Output weight matrix V can be updated in a simpler manner

$$\frac{\partial L_i}{\partial V} = \sum_{i=1}^t \left(\frac{\partial L}{\partial V} \right)_i = \sum_{i=1}^t \left[\frac{\partial L}{\partial y_i} \cdot \frac{\partial y_i}{\partial V} \right]$$

Other Weight Matrices

- Input weight matrix U is updated in the same way

$$\frac{\partial L}{\partial U} = \sum_{i=1}^t \left[\frac{\partial L_i}{\partial y_i} \cdot \frac{\partial y_i}{\partial h_i} \cdot \left(\prod_{j=t}^{i-1} \frac{\partial h_j}{\partial h_{j-1}} \right) \cdot \frac{\partial h_i}{\partial U} \right]$$

- Output weight matrix V can be updated in a simpler manner

$$\frac{\partial L_i}{\partial V} = \sum_{i=1}^t \left(\frac{\partial L}{\partial V} \right)_i = \sum_{i=1}^t \left[\frac{\partial L}{\partial y_i} \cdot \frac{\partial y_i}{\partial V} \right]$$

- Bias can be treated separately or uniformly by putting inside the weight matrices

Problems of RNNs

- Suffers from **short-term memory**
 - May leave out important information that is far behind (e.g., at the beginning of a sentence)

Problems of RNNs

- Suffers from **short-term memory**
 - May leave out important information that is far behind (e.g., at the beginning of a sentence)
- Solution: Learn to keep or ignore information in a long sequence

Problems of RNNs

- Suffers from **short-term memory**
 - May leave out important information that is far behind (e.g., at the beginning of a sentence)
- Solution: Learn to keep or ignore information in a long sequence
- **Vanishing gradients**
 - Gradients are multiplied by the chain rule
 - Each gradient is a number between 0 to 1, or -1 to 1
 - Thus, for long sequences, gradients become too small
 - No learning

Problems of RNNs

- Suffers from **short-term memory**
 - May leave out important information that is far behind (e.g., at the beginning of a sentence)
- Solution: Learn to keep or ignore information in a long sequence
- **Vanishing gradients**
 - Gradients are multiplied by the chain rule
 - Each gradient is a number between 0 to 1, or -1 to 1
 - Thus, for long sequences, gradients become too small
 - No learning
- **Exploding gradients**
 - For gradients that are greater than 1 or less than -1

Problems of RNNs

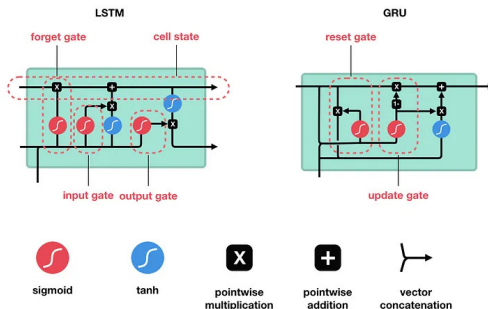
- Suffers from **short-term memory**
 - May leave out important information that is far behind (e.g., at the beginning of a sentence)
- Solution: Learn to keep or ignore information in a long sequence
- **Vanishing gradients**
 - Gradients are multiplied by the chain rule
 - Each gradient is a number between 0 to 1, or -1 to 1
 - Thus, for long sequences, gradients become too small
 - No learning
- **Exploding gradients**
 - For gradients that are greater than 1 or less than -1
- Solution: Neural network structure instead of a single activation function

Variants of RNNs

- Most famous variant is **LSTM (Long Short-Term Memory)**
 - *Forget gate* that keeps/ignores information
 - *Input gate* to decide which information from current state to process
 - *Cell state* holds current information
 - *Output gate* to pass on information to the next layer

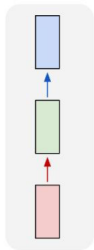
Variants of RNNs

- Most famous variant is **LSTM (Long Short-Term Memory)**
 - *Forget gate* that keeps/ignores information
 - *Input gate* to decide which information from current state to process
 - *Cell state* holds current information
 - *Output gate* to pass on information to the next layer
- A simpler variant is **GRU (Gated Recurrent Unit)**
 - *Update gate* decides what information to process
 - *Reset gate* decides how much of the past information to forget

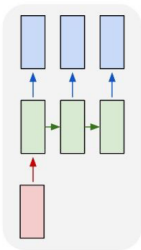


Uses of RNNs

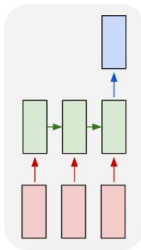
one to one



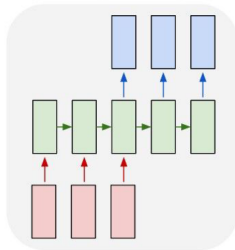
one to many



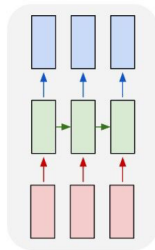
many to one



many to many



many to many



- Multiple types of RNNs for multiple types of use-cases

Convolutional Neural Networks

- **Convolutional Neural Networks (CNNs)** approximate fully connected feed-forward networks

Convolutional Neural Networks

- **Convolutional Neural Networks (CNNs)** approximate fully connected feed-forward networks
- Especially useful for images
- Full connections are too many in number and require impractical training size

Convolutional Neural Networks

- **Convolutional Neural Networks (CNNs)** approximate fully connected feed-forward networks
- Especially useful for images
- Full connections are too many in number and require impractical training size
- **Convolutional layer** combines *spatially local* neurons into one output neuron in the next layer
 - Tiling of 4×4 pixels into 1 value requires learning only 16 weights

Convolutional Neural Networks

- **Convolutional Neural Networks (CNNs)** approximate fully connected feed-forward networks
- Especially useful for images
- Full connections are too many in number and require impractical training size
- **Convolutional layer** combines *spatially local* neurons into one output neuron in the next layer
 - Tiling of 4×4 pixels into 1 value requires learning only 16 weights
- **Pooling** layer combines several input neurons into one output neuron in the next layer
 - Common is *max pooling*

Convolutional Neural Networks

- **Convolutional Neural Networks (CNNs)** approximate fully connected feed-forward networks
- Especially useful for images
- Full connections are too many in number and require impractical training size
- **Convolutional layer** combines *spatially local* neurons into one output neuron in the next layer
 - Tiling of 4×4 pixels into 1 value requires learning only 16 weights
- **Pooling** layer combines several input neurons into one output neuron in the next layer
 - Common is *max pooling*
- Activation function is often simplified to **ReLU (rectified linear units)**:
 $f(x) = \max\{0, x\}$

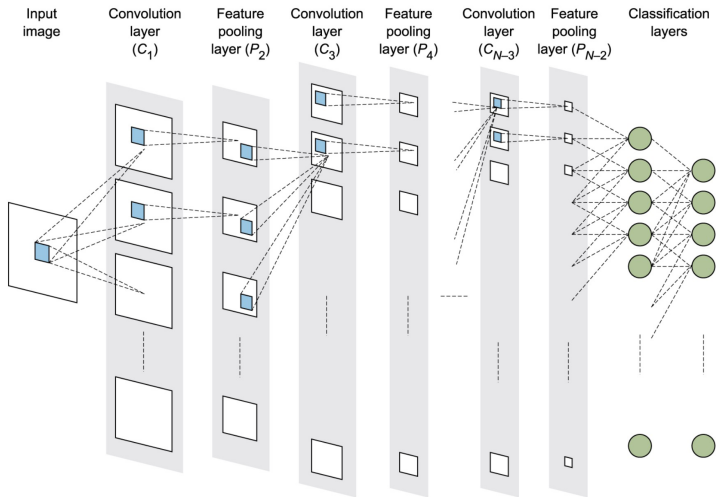
Convolutional Neural Networks

- **Convolutional Neural Networks (CNNs)** approximate fully connected feed-forward networks
- Especially useful for images
- Full connections are too many in number and require impractical training size
- **Convolutional layer** combines *spatially local* neurons into one output neuron in the next layer
 - Tiling of 4×4 pixels into 1 value requires learning only 16 weights
- **Pooling** layer combines several input neurons into one output neuron in the next layer
 - Common is *max pooling*
- Activation function is often simplified to **ReLU (rectified linear units)**:
 $f(x) = \max\{0, x\}$
 - Relaxed version is **Leaky ReLU**

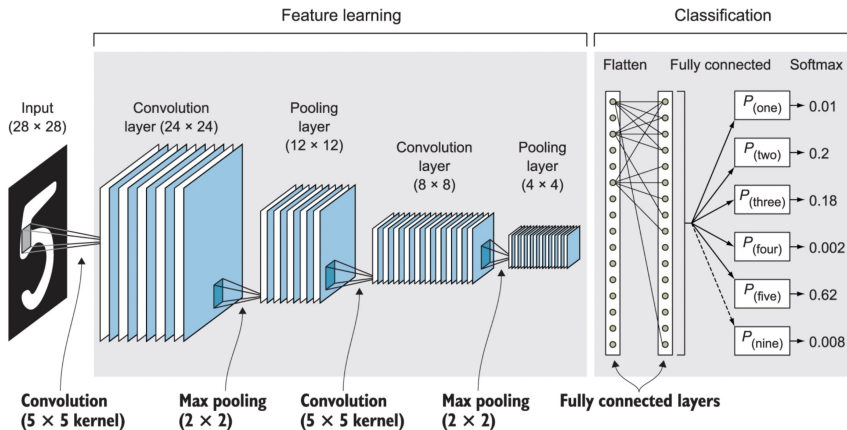
Convolutional Neural Networks

- **Convolutional Neural Networks (CNNs)** approximate fully connected feed-forward networks
- Especially useful for images
- Full connections are too many in number and require impractical training size
- **Convolutional layer** combines *spatially local* neurons into one output neuron in the next layer
 - Tiling of 4×4 pixels into 1 value requires learning only 16 weights
- **Pooling** layer combines several input neurons into one output neuron in the next layer
 - Common is *max pooling*
- Activation function is often simplified to **ReLU (rectified linear units)**:
 $f(x) = \max\{0, x\}$
 - Relaxed version is **Leaky ReLU**
- Includes *fully connected layers (dense layers)* as well

CNN Architecture



CNN Example



CNN Operations

- **Convolution** uses a *filter* (or *kernel*)
- Kernel is applied over a region and slided over all regions

CNN Operations

- **Convolution** uses a *filter* (or *kernel*)
- Kernel is applied over a region and slid over all regions
- For image I and kernel K , feature map S is

$$S(i, j) = (I * K)(i, j) = \sum_{x=0, y=0}^{p-1, q-1} [I(i+x, j+y) \cdot K(x, y)]$$

CNN Operations

- **Convolution** uses a *filter* (or *kernel*)
- Kernel is applied over a region and slid over all regions
- For image I and kernel K , feature map S is

$$S(i, j) = (I * K)(i, j) = \sum_{x=0, y=0}^{p-1, q-1} [I(i+x, j+y) \cdot K(x, y)]$$

- **Padding** may be done if needed to preserve size of object

CNN Operations

- **Convolution** uses a *filter* (or *kernel*)
- Kernel is applied over a region and slid over all regions
- For image I and kernel K , feature map S is

$$S(i, j) = (I * K)(i, j) = \sum_{x=0, y=0}^{p-1, q-1} [I(i+x, j+y) \cdot K(x, y)]$$

- **Padding** may be done if needed to preserve size of object
- **Stride** controls how much to slide
 - Has an effect on output size

CNN Operations

- **Convolution** uses a *filter* (or *kernel*)
- Kernel is applied over a region and slid over all regions
- For image I and kernel K , feature map S is

$$S(i, j) = (I * K)(i, j) = \sum_{x=0, y=0}^{p-1, q-1} [I(i+x, j+y) \cdot K(x, y)]$$

- **Padding** may be done if needed to preserve size of object
- **Stride** controls how much to slide
 - Has an effect on output size
- **Pooling** combines information from neighbors
 - Similar to *convolution*, but without a fixed kernel
 - *Max pooling* is

$$P(i, j) = (I * M)(i, j) = \max_{x=0, y=0}^{p-1, q-1} [I(i+x, j+y)]$$

CNN in Text

- CNN has been adopted for text

- CNN has been adopted for text
- Text is one-dimensional
- Sliding window of a particular length over the text
- Each window captures a fixed number of tokens
- Stride determines how close the windows are
- Stride of 1 captures all sliding windows
- Stride of width of window makes all windows independent

- CNN has been adopted for text
- Text is one-dimensional
- Sliding window of a particular length over the text
- Each window captures a fixed number of tokens
- Stride determines how close the windows are
- Stride of 1 captures all sliding windows
- Stride of width of window makes all windows independent
- Can stack multiple layers